```python
import kagglehub

# Download latest version
path = kagglehub.dataset_download("rifat963/dhakaai-dhaka-based-traffic-detection-dataset")

print("Path to dataset files:", path)
```

```
Path to dataset files: /kaggle/input/datasets/rifat963/dhakaai-dhaka-based-traffic-detection-dataset
```

```python
import os

def list_files(startpath):
    for root, dirs, files in os.walk(startpath):
        level = root.replace(startpath, '').count(os.sep)
        indent = ' ' * 4 * (level)
        print(f'{indent}{os.path.basename(root)}/')
        subindent = ' ' * 4 * (level + 1)
        # Show only first 3 files to avoid flooding the screen
        for f in files[:3]:
            print(f'{subindent}{f}')

# Use the path variable from your previous cell
list_files(path)
```

```
dhakaai-dhaka-based-traffic-detection-dataset/
    test round 2/
        test round 2/
            sabiha(115)_jpg.rf.30b61b8c14566d5002cd5db54e17e771.jpg
            Pias (653)_jpg.rf.dfa42cc9725480c6d524b58d21318686.jpg
            Pias (603)_jpg.rf.67686941a89fc127f0184443318fbb54.jpg
    test/
        test/
            Sabiha_(4)_jpg.rf.134cf55311b25147f3238699e22ac08d.jpg
            sabiha(234)_jpg.rf.0b4a7fb02b589761a96d5918ecef8239.jpg
            Shykat_02_053_jpg.rf.6c7beca35c4817cc97a42d7086b33669.jpg
    train/
        Final Train Dataset/
            Navid_105.jpg
            Asraf_15.jpg
            Dipto_579.jpg
```

```python
import os
import glob

# Set the path to the training folder based on your output
train_dir = os.path.join(path, "train/Final Train Dataset")

# Count different file types in that specific folder
image_files = glob.glob(os.path.join(train_dir, "*.jpg"))
```

```python
txt_files = glob.glob(os.path.join(train_dir, "*.txt"))
xml_files = glob.glob(os.path.join(train_dir, "*.xml"))
json_files = glob.glob(os.path.join(train_dir, "*.json"))

print("--- DIRECTORY SCAN RESULTS ---")
print(f"Total Images (.jpg): {len(image_files)}")
print(f"Total TXT labels: {len(txt_files)}")
print(f"Total XML labels: {len(xml_files)}")
print(f"Total JSON labels: {len(json_files)}")
```

```
--- DIRECTORY SCAN RESULTS ---
Total Images (.jpg): 2844
Total TXT labels: 0
Total XML labels: 3003
Total JSON labels: 0
```

```python
import os
import glob
import xml.etree.ElementTree as ET
import shutil

# 1. Define our paths
source_dir = os.path.join(path, "train/Final Train Dataset")
work_dir = "/kaggle/working/dhaka_yolo"

images_dir = os.path.join(work_dir, "images/train")
labels_dir = os.path.join(work_dir, "labels/train")

os.makedirs(images_dir, exist_ok=True)
os.makedirs(labels_dir, exist_ok=True)

print("Scanning for classes (skipping corrupted files)...")
classes = set()
xml_files = glob.glob(os.path.join(source_dir, "*.xml"))

# Safely extract classes
for xml_file in xml_files:
    try:
        tree = ET.parse(xml_file)
        root = tree.getroot()
        for obj in root.findall('object'):
            name_elem = obj.find('name')
            if name_elem is not None and name_elem.text:
                classes.add(name_elem.text)
    except ET.ParseError:
        # If the file is broken, we silently ignore it and continue
        continue

# Sort alphabetically
classes = sorted(list(classes))
```

```python
print(f"\nFound {len(classes)} unique classes:")
for i, cls in enumerate(classes):
    print(f"ID {i}: {cls}")

# 3. The Math Function
def convert_bbox(size, box):
    dw = 1. / size[0]
    dh = 1. / size[1]
    x = (box[0] + box[1]) / 2.0
    y = (box[2] + box[3]) / 2.0
    w = box[1] - box[0]
    h = box[3] - box[2]
    return (x * dw, y * dh, w * dw, h * dh)

print("\nStarting conversion and copying to /kaggle/working/...")
image_files = glob.glob(os.path.join(source_dir, "*.jpg"))
copied_count = 0
skipped_count = 0

# 4. Safely loop through images and convert
for img_path in image_files:
    basename = os.path.basename(img_path)
    name_without_ext = os.path.splitext(basename)[0]
    xml_path = os.path.join(source_dir, name_without_ext + ".xml")

    if os.path.exists(xml_path):
        try:
            tree = ET.parse(xml_path)
            root = tree.getroot()
            size = root.find('size')

            if size is None or size.find('width') is None:
                skipped_count += 1
                continue

            w = int(size.find('width').text)
            h = int(size.find('height').text)

            if w == 0 or h == 0:
                skipped_count += 1
                continue

            txt_path = os.path.join(labels_dir, name_without_ext +
".txt")
            with open(txt_path, "w") as out_file:
                for obj in root.findall('object'):
                    cls_name = obj.find('name').text
                    if cls_name not in classes:
                        continue
                    cls_id = classes.index(cls_name)
```

```
                    xmlbox = obj.find('bndbox')

                    b = (float(xmlbox.find('xmin').text),
float(xmlbox.find('xmax').text),
                         float(xmlbox.find('ymin').text),
float(xmlbox.find('ymax').text))

                    bb = convert_bbox((w, h), b)
                    out_file.write(f"{cls_id} {' '.join([str(a) for a
in bb])}\n")

            shutil.copy(img_path, os.path.join(images_dir, basename))
            copied_count += 1

        except ET.ParseError:
            skipped_count += 1
            continue

print(f"\nSUCCESS! Converted and copied {copied_count} image-label
pairs.")
print(f"Skipped {skipped_count} broken/unmatched files.")
```

```
Scanning for classes (skipping corrupted files)...

Found 21 unique classes:
ID 0: ambulance
ID 1: army vehicle
ID 2: auto rickshaw
ID 3: bicycle
ID 4: bus
ID 5: car
ID 6: garbagevan
ID 7: human hauler
ID 8: minibus
ID 9: minivan
ID 10: motorbike
ID 11: pickup
ID 12: policecar
ID 13: rickshaw
ID 14: scooter
ID 15: suv
ID 16: taxi
ID 17: three wheelers (CNG)
ID 18: truck
ID 19: van
ID 20: wheelbarrow

Starting conversion and copying to /kaggle/working/...
```

```
SUCCESS! Converted and copied 2843 image-label pairs.
Skipped 1 broken/unmatched files.
```

```python
import yaml
import os

# The exact 21 classes we just extracted
classes = [
    'ambulance', 'army vehicle', 'auto rickshaw', 'bicycle', 'bus',
    'car', 'garbagevan', 'human hauler', 'minibus', 'minivan',
    'motorbike', 'pickup', 'policecar', 'rickshaw', 'scooter',
    'suv', 'taxi', 'three wheelers (CNG)', 'truck', 'van',
'wheelbarrow'
]

# Create the YAML dictionary
yaml_data = {
    'path': '/kaggle/working/dhaka_yolo', # Root directory
    'train': 'images/train',              # Training images
    'val': 'images/train',                # Validation images (reusing
train for this initial test)
    'nc': len(classes),                   # Number of classes
    'names': classes                      # Class names
}

# Write to a file
yaml_file_path = '/kaggle/working/dhaka_traffic.yaml'
with open(yaml_file_path, 'w') as f:
    yaml.dump(yaml_data, f, sort_keys=False)

print(f"Successfully created YAML file at: {yaml_file_path}")
```

```
Successfully created YAML file at: /kaggle/working/dhaka_traffic.yaml
```

```python
import os
import random
import shutil
import yaml

# 1. Setup Directories
base_dir = '/kaggle/working/dhaka_yolo'
train_img_dir = os.path.join(base_dir, 'images/train')
train_lbl_dir = os.path.join(base_dir, 'labels/train')

val_img_dir = os.path.join(base_dir, 'images/val')
val_lbl_dir = os.path.join(base_dir, 'labels/val')

os.makedirs(val_img_dir, exist_ok=True)
os.makedirs(val_lbl_dir, exist_ok=True)

# 2. Get all current training images
```

```
all_images = [f for f in os.listdir(train_img_dir) if
f.endswith('.jpg')]

# 3. SHUFFLE the data randomly
random.seed(42) # Keeps the randomness consistent
random.shuffle(all_images)

# 4. Calculate the 20% split
split_index = int(len(all_images) * 0.8)
train_files = all_images[:split_index]
val_files = all_images[split_index:]

print(f"Total images: {len(all_images)}")
print(f"Keeping {len(train_files)} for Training (80%)")
print(f"Moving {len(val_files)} for Validation (20%)")

# 5. Move the 20% Validation files (Images and matching Labels)
moved_count = 0
for img_name in val_files:
    # Move image
    src_img = os.path.join(train_img_dir, img_name)
    dst_img = os.path.join(val_img_dir, img_name)
    shutil.move(src_img, dst_img)

    # Move matching label
    lbl_name = img_name.replace('.jpg', '.txt')
    src_lbl = os.path.join(train_lbl_dir, lbl_name)
    dst_lbl = os.path.join(val_lbl_dir, lbl_name)

    if os.path.exists(src_lbl):
        shutil.move(src_lbl, dst_lbl)
        moved_count += 1

print(f"Successfully moved {moved_count} image-label pairs to
Validation set.")

# 6. Update the YAML file with the correct paths
classes = [
    'ambulance', 'army vehicle', 'auto rickshaw', 'bicycle', 'bus',
    'car', 'garbagevan', 'human hauler', 'minibus', 'minivan',
    'motorbike', 'pickup', 'policecar', 'rickshaw', 'scooter',
    'suv', 'taxi', 'three wheelers (CNG)', 'truck', 'van',
'wheelbarrow'
]

yaml_data = {
    'path': '/kaggle/working/dhaka_yolo',
    'train': 'images/train',              # Now strictly 80%
    'val': 'images/val',                  # Now strictly 20% unseen
data
```

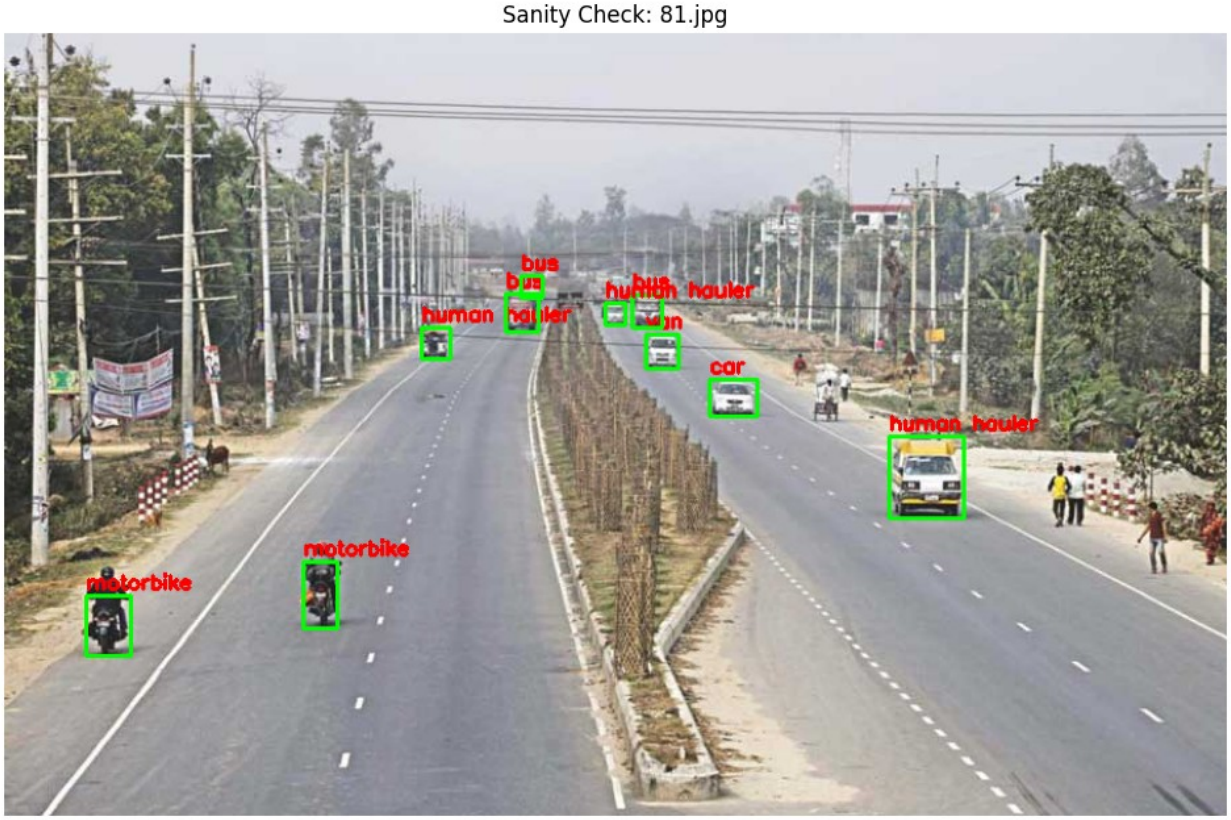
```
    'nc': len(classes),
    'names': classes
}

yaml_file_path = '/kaggle/working/dhaka_traffic.yaml'
with open(yaml_file_path, 'w') as f:
    yaml.dump(yaml_data, f, sort_keys=False)

print("YAML file updated with correct Train/Val split!")
```

```
Total images: 2274
Keeping 1819 for Training (80%)
Moving 455 for Validation (20%)
Successfully moved 455 image-label pairs to Validation set.
YAML file updated with correct Train/Val split!
```

```
import cv2
import matplotlib.pyplot as plt
import os
import random

# 1. Define paths and classes
img_dir = '/kaggle/working/dhaka_yolo/images/train'
lbl_dir = '/kaggle/working/dhaka_yolo/labels/train'
classes = [
    'ambulance', 'army vehicle', 'auto rickshaw', 'bicycle', 'bus',
    'car', 'garbagevan', 'human hauler', 'minibus', 'minivan',
    'motorbike', 'pickup', 'policecar', 'rickshaw', 'scooter',
    'suv', 'taxi', 'three wheelers (CNG)', 'truck', 'van',
'wheelbarrow'
]

# 2. Pick a random image
all_images = [f for f in os.listdir(img_dir) if f.endswith('.jpg')]
random_img = random.choice(all_images)
img_path = os.path.join(img_dir, random_img)
lbl_path = os.path.join(lbl_dir, random_img.replace('.jpg', '.txt'))

# 3. Load the image
img = cv2.imread(img_path)
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
h, w, _ = img.shape

# 4. Read the labels and draw boxes
if os.path.exists(lbl_path):
    with open(lbl_path, 'r') as f:
        for line in f.readlines():
            parts = line.strip().split()
            cls_id = int(parts[0])
            x_center, y_center, width, height = map(float, parts[1:])
```

```
            # Reversing the YOLO math: Percentages back to pixel
coordinates
            x1 = int((x_center - width / 2) * w)
            y1 = int((y_center - height / 2) * h)
            x2 = int((x_center + width / 2) * w)
            y2 = int((y_center + height / 2) * h)

            # Draw the rectangle
            cv2.rectangle(img, (x1, y1), (x2, y2), (0, 255, 0), 2)

            # Add the class name text
            label = classes[cls_id]
            cv2.putText(img, label, (x1, y1 - 5),
cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 0, 0), 2)

# 5. Show the image
plt.figure(figsize=(12, 8))
plt.imshow(img)
plt.axis('off')
plt.title(f"Sanity Check: {random_img}")
plt.show()
```

Sanity Check: 81.jpg



```python
# Install the Ultralytics library if you haven't already
!pip install ultralytics -q

from ultralytics import YOLO

# 1. Load the pre-trained YOLOv9 "Compact" model
model = YOLO('yolov9c.pt')

# 2. Train the model using BOTH GPUs
print("Starting official training on Dual T4 GPUs for 20 epochs...")
results = model.train(
    data='/kaggle/working/dhaka_traffic.yaml', # Points to our updated
YAML
    epochs=20,
    imgsz=640,
    batch=32,
    device=[0, 1],      # Engages both T4 GPUs
    project='/kaggle/working/runs',
    name='dhaka_traffic_model',
    verbose=True
)
```

```
Starting official training on Dual T4 GPUs for 20 epochs...
Ultralytics 8.4.21 □ Python-3.12.12 torch-2.9.0+cu126 CUDA:0 (Tesla
T4, 14913MiB)
                                                        CUDA:1 (Tesla
T4, 14913MiB)
engine/trainer: agnostic_nms=False, amp=True, angle=1.0,
augment=False, auto_augment=randaugment, batch=32, bgr=0.0, box=7.5,
cache=False, cfg=None, classes=None, close_mosaic=10, cls=0.5,
compile=False, conf=None, copy_paste=0.0, copy_paste_mode=flip,
cos_lr=False, cutmix=0.0, data=/kaggle/working/dhaka_traffic.yaml,
degrees=0.0, deterministic=True, device=0,1, dfl=1.5, dnn=False,
dropout=0.0, dynamic=False, embed=None, end2end=None, epochs=20,
erasing=0.4, exist_ok=False, fliplr=0.5, flipud=0.0,
format=torchscript, fraction=1.0, freeze=None, half=False,
hsv_h=0.015, hsv_s=0.7, hsv_v=0.4, imgsz=640, int8=False, iou=0.7,
keras=False, kobj=1.0, line_width=None, lr0=0.01, lrf=0.01,
mask_ratio=4, max_det=300, mixup=0.0, mode=train, model=yolov9c.pt,
momentum=0.937, mosaic=1.0, multi_scale=0.0,
name=dhaka_traffic_model2, nbs=64, nms=False, opset=None,
optimize=False, optimizer=auto, overlap_mask=True, patience=100,
perspective=0.0, plots=True, pose=12.0, pretrained=True,
profile=False, project=/kaggle/working/runs, rect=False, resume=False,
retina_masks=False, rle=1.0, save=True, save_conf=False,
save_crop=False, save_dir=/kaggle/working/runs/dhaka_traffic_model2,
save_frames=False, save_json=False, save_period=-1, save_txt=False,
scale=0.5, seed=0, shear=0.0, show=False, show_boxes=True,
show_conf=True, show_labels=True, simplify=True, single_cls=False,
source=None, split=val, stream_buffer=False, task=detect, time=None,
```

```
tracker=botsort.yaml, translate=0.1, val=True, verbose=True,
vid_stride=1, visualize=False, warmup_bias_lr=0.1, warmup_epochs=3.0,
warmup_momentum=0.8, weight_decay=0.0005, workers=8, workspace=None
Overriding model.yaml nc=80 with nc=21

                   from  n    params  module
arguments
  0                  -1  1      1856  ultralytics.nn.modules.conv.Conv
[3, 64, 3, 2]
  1                  -1  1     73984  ultralytics.nn.modules.conv.Conv
[64, 128, 3, 2]
  2                  -1  1    212864
ultralytics.nn.modules.block.RepNCSPELAN4    [128, 256, 128, 64, 1]

  3                  -1  1    164352
ultralytics.nn.modules.block.ADown           [256, 256]

  4                  -1  1    847616
ultralytics.nn.modules.block.RepNCSPELAN4    [256, 512, 256, 128, 1]

  5                  -1  1    656384
ultralytics.nn.modules.block.ADown           [512, 512]

  6                  -1  1   2857472
ultralytics.nn.modules.block.RepNCSPELAN4    [512, 512, 512, 256, 1]

  7                  -1  1    656384
ultralytics.nn.modules.block.ADown           [512, 512]

  8                  -1  1   2857472
ultralytics.nn.modules.block.RepNCSPELAN4    [512, 512, 512, 256, 1]

  9                  -1  1    656896
ultralytics.nn.modules.block.SPPELAN         [512, 512, 256]

 10                  -1  1         0
torch.nn.modules.upsampling.Upsample         [None, 2, 'nearest']

 11             [-1, 6]  1         0
ultralytics.nn.modules.conv.Concat           [1]

 12                  -1  1   3119616
ultralytics.nn.modules.block.RepNCSPELAN4    [1024, 512, 512, 256, 1]

 13                  -1  1         0
torch.nn.modules.upsampling.Upsample         [None, 2, 'nearest']

 14             [-1, 4]  1         0
ultralytics.nn.modules.conv.Concat           [1]

 15                  -1  1    912640
```

```
ultralytics.nn.modules.block.RepNCSPELAN4    [1024, 256, 256, 128, 1]

 16                  -1  1    164352
ultralytics.nn.modules.block.ADown           [256, 256]

 17            [-1, 12]  1         0
ultralytics.nn.modules.conv.Concat           [1]

 18                  -1  1   2988544
ultralytics.nn.modules.block.RepNCSPELAN4    [768, 512, 512, 256, 1]

 19                  -1  1    656384
ultralytics.nn.modules.block.ADown           [512, 512]

 20             [-1, 9]  1         0
ultralytics.nn.modules.conv.Concat           [1]

 21                  -1  1   3119616
ultralytics.nn.modules.block.RepNCSPELAN4    [1024, 512, 512, 256, 1]

 22        [15, 18, 21]  1   5598991
ultralytics.nn.modules.head.Detect           [21, 16, None, [256, 512,
512]]
YOLOv9c summary: 358 layers, 25,545,423 parameters, 25,545,407
gradients, 103.8 GFLOPs

Transferred 931/937 items from pretrained weights
DDP: debug command /usr/bin/python3 -m torch.distributed.run --
nproc_per_node 2 --master_port 47523
/root/.config/Ultralytics/DDP/_temp_ii10zerk134131003824352.py
Ultralytics 8.4.21 □ Python-3.12.12 torch-2.9.0+cu126 CUDA:0 (Tesla
T4, 14913MiB)
                                                         CUDA:1 (Tesla
T4, 14913MiB)
Overriding model.yaml nc=80 with nc=21
Transferred 931/937 items from pretrained weights
Freezing layer 'model.22.dfl.conv.weight'
AMP: running Automatic Mixed Precision (AMP) checks...
AMP: checks passed □
train: Fast image access □ (ping: 0.0±0.0 ms, read: 1785.0±841.6 MB/s,
size: 274.5 KB)
train: Scanning /kaggle/working/dhaka_yolo/labels/train... 1819
images, 0 backgrounds, 0 corrupt: 100% ━━━━━━━━━━━━ 1819/1819 1.3Kit/s
1.4s0.1s
train: New cache created:
/kaggle/working/dhaka_yolo/labels/train.cache
albumentations: Blur(p=0.01, blur_limit=(3, 7)), MedianBlur(p=0.01,
blur_limit=(3, 7)), ToGray(p=0.01, method='weighted_average',
num_output_channels=3), CLAHE(p=0.01, clip_limit=(1.0, 4.0),
tile_grid_size=(8, 8))
```

```
val: Fast image access  (ping: 0.0±0.0 ms, read: 1362.1±641.9 MB/s,
size: 225.7 KB)
val: Scanning /kaggle/working/dhaka_yolo/labels/val... 1024 images, 0
backgrounds, 0 corrupt: 100% ━━━━━━━━━━━━ 1024/1024 1.6Kit/s 0.6s0.0s
val: New cache created: /kaggle/working/dhaka_yolo/labels/val.cache
optimizer: 'optimizer=auto' found, ignoring 'lr0=0.01' and
'momentum=0.937' and determining best 'optimizer', 'lr0' and
'momentum' automatically...
optimizer: AdamW(lr=0.0004, momentum=0.9) with parameter groups 154
weight(decay=0.0), 161 weight(decay=0.0005), 160 bias(decay=0.0)
Plotting labels to
/kaggle/working/runs/dhaka_traffic_model2/labels.jpg...
Image sizes 640 train, 640 val
Using 4 dataloader workers
Logging results to /kaggle/working/runs/dhaka_traffic_model2
Starting training for 20 epochs...

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 16/16 1.9it/s 8.4s0.5s
                   all       1024       8314      0.535      0.254
0.222      0.139

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 16/16 2.2it/s 7.3s0.5s
                   all       1024       8314      0.493      0.255
0.253      0.161

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 16/16 2.1it/s 7.6s0.5s
                   all       1024       8314      0.485      0.288
0.281      0.176

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 16/16 2.2it/s 7.3s0.5s
                   all       1024       8314      0.612      0.278
0.29      0.186

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 16/16 2.2it/s 7.2s0.5s
                   all       1024       8314      0.562      0.315
```

```
0.311      0.193

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 16/16 2.1it/s 7.6s0.5s
                   all       1024       8314      0.473      0.359
0.34      0.219

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 16/16 2.1it/s 7.6s0.5s
                   all       1024       8314      0.466      0.375
0.382       0.25

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 16/16 2.0it/s 7.8s0.5s
                   all       1024       8314      0.506      0.381
0.378       0.25

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 16/16 2.2it/s 7.4s0.5s
                   all       1024       8314      0.607      0.365
0.395      0.262

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 16/16 2.2it/s 7.3s0.5s
                   all       1024       8314       0.61      0.373
0.418      0.275
Closing dataloader mosaic
albumentations: Blur(p=0.01, blur_limit=(3, 7)), MedianBlur(p=0.01,
blur_limit=(3, 7)), ToGray(p=0.01, method='weighted_average',
num_output_channels=3), CLAHE(p=0.01, clip_limit=(1.0, 4.0),
tile_grid_size=(8, 8))

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 16/16 2.2it/s 7.3s0.5s
                   all       1024       8314      0.615      0.381
0.398      0.254

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
```

```
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 16/16 2.2it/s 7.3s0.5s
                   all       1024       8314        0.6      0.397
0.424       0.28

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 16/16 2.2it/s 7.3s0.5s
                   all       1024       8314      0.518      0.405
0.425      0.282

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 16/16 2.2it/s 7.3s0.5s
                   all       1024       8314      0.656      0.393
0.433      0.288

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 16/16 2.3it/s 7.0s0.4s
                   all       1024       8314      0.594      0.404
0.446      0.301

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 16/16 2.2it/s 7.4s0.5s
                   all       1024       8314      0.661      0.407
0.459      0.307

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 16/16 2.2it/s 7.4s0.5s
                   all       1024       8314      0.542      0.422
0.459      0.308

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 16/16 2.2it/s 7.4s0.5s
                   all       1024       8314      0.589      0.431
0.465      0.313

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
```

```
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━━━━━━━━━ 16/16 2.2it/s 7.1s0.4s
                   all       1024       8314      0.627       0.43
0.471      0.319

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━━━━━━━━━ 16/16 2.2it/s 7.3s0.5s
                   all       1024       8314      0.546      0.464
0.474      0.321

20 epochs completed in 0.360 hours.
Optimizer stripped from
/kaggle/working/runs/dhaka_traffic_model2/weights/last.pt, 51.6MB
Optimizer stripped from
/kaggle/working/runs/dhaka_traffic_model2/weights/best.pt, 51.6MB

Validating
/kaggle/working/runs/dhaka_traffic_model2/weights/best.pt...
YOLOv9c summary (fused): 156 layers, 25,335,439 parameters, 0
gradients, 102.4 GFLOPs
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━━━━━━━━━ 16/16 1.8it/s 9.0s0.5s
                   all       1024       8314      0.547      0.464
0.474      0.321
             ambulance         22         23      0.352      0.174
0.223      0.123
          army vehicle         15         16          1      0.611
0.674      0.548
         auto rickshaw         61        159      0.754      0.528
0.64      0.415
               bicycle        118        177      0.502      0.446
0.418        0.2
                   bus        550       1180      0.767      0.774
0.805      0.571
                   car        532       1820      0.714      0.781
0.788      0.555
            garbagevan          1          1          1          0
0          0
          human hauler         39         42      0.492      0.476
0.462      0.286
               minibus         16         22      0.268      0.182
0.107     0.0547
               minivan        202        318      0.441      0.443
0.418      0.311
             motorbike        407        776      0.674      0.718
0.723      0.391
                pickup        284        437      0.565      0.568
0.554      0.379
```

```
              policecar         12         12          0          0
0.0194    0.00919
               rickshaw        340       1223      0.664      0.679
0.708      0.448
                scooter          9         10          0          0
0.0196     0.0087
                    suv        170        271      0.427      0.464
0.428      0.312
                   taxi         20         23      0.525      0.609
0.633      0.492
  three wheelers (CNG)        403        985      0.792      0.775
0.822      0.553
                  truck        305        529      0.718      0.709
0.752      0.541
                    van        146        250      0.486      0.476
0.441      0.318
            wheelbarrow         34         40      0.357      0.325
0.315      0.217
Speed: 0.1ms preprocess, 4.4ms inference, 0.0ms loss, 0.9ms
postprocess per image
Results saved to /kaggle/working/runs/dhaka_traffic_model2
```

```python
import os
import random
import kagglehub
from ultralytics import YOLO
import matplotlib.pyplot as plt
import cv2

# 1. Ensure we have the correct dynamic path to the dataset
path = kagglehub.dataset_download("rifat963/dhakaai-dhaka-based-
traffic-detection-dataset")

# 2. Load YOUR custom trained brain
model_path =
'/kaggle/working/runs/dhaka_traffic_model2/weights/best.pt'
custom_model = YOLO(model_path)

# 3. Pick a random test image using the dynamic path!
test_img_dir = os.path.join(path, 'test/test')
test_images = [f for f in os.listdir(test_img_dir) if
f.endswith('.jpg')]
test_img_path = os.path.join(test_img_dir, random.choice(test_images))

# 4. Make the Prediction
print(f"Running prediction on: {os.path.basename(test_img_path)}")
results = custom_model.predict(source=test_img_path, conf=0.25,
save=True, project='/kaggle/working/predictions', name='test_results',
exist_ok=True)
```

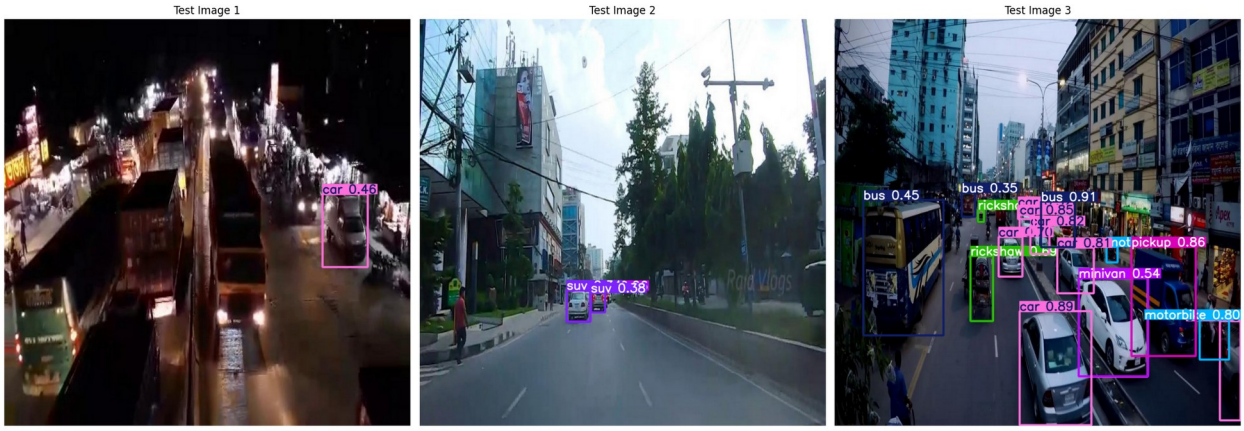
```python
# 5. Display the Result
result_img_path =
os.path.join('/kaggle/working/predictions/test_results',
os.path.basename(test_img_path))
img = cv2.imread(result_img_path)
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

plt.figure(figsize=(12, 12))
plt.imshow(img)
plt.axis('off')
plt.title("Final AI Prediction on Unseen Dhaka Traffic")
plt.show()

# 6. Extract the Counts (The exact input for our RL Agent)
print("\n--- VEHICLE COUNTS FOR RL AGENT ---")
counts = {}
for r in results:
    for c in r.boxes.cls:
        class_name = custom_model.names[int(c)]
        counts[class_name] = counts.get(class_name, 0) + 1

if not counts:
    print("No vehicles detected with >25% confidence in this image.")
else:
    for vehicle, count in counts.items():
        print(f"{vehicle}: {count}")
```

```
Running prediction on:
sabiha(202)_jpg.rf.1fa2edf1c1750735ec6acd777c799665.jpg

image 1/1 /kaggle/input/datasets/rifat963/dhakaai-dhaka-based-traffic-
detection-dataset/test/test/
sabiha(202)_jpg.rf.1fa2edf1c1750735ec6acd777c799665.jpg: 640x640 1
three wheelers (CNG), 35.6ms
Speed: 3.6ms preprocess, 35.6ms inference, 20.0ms postprocess per
image at shape (1, 3, 640, 640)
Results saved to /kaggle/working/predictions/test_results
```

Final AI Prediction on Unseen Dhaka Traffic

```
--- VEHICLE COUNTS FOR RL AGENT ---
three wheelers (CNG): 1

import os
import random
import cv2
import matplotlib.pyplot as plt
from ultralytics import YOLO
import kagglehub

# 1. Paths and Model
```

```
path = kagglehub.dataset_download("rifat963/dhakaai-dhaka-based-traffic-detection-dataset")
model_path = '/kaggle/working/runs/dhaka_traffic_model2/weights/best.pt'
custom_model = YOLO(model_path)

# 2. Pick 3 random images
test_img_dir = os.path.join(path, 'test/test')
test_images = [f for f in os.listdir(test_img_dir) if f.endswith('.jpg')]
random_samples = random.sample(test_images, 3)

# 3. Predict and Plot side-by-side
plt.figure(figsize=(20, 8))
for i, img_name in enumerate(random_samples):
    img_path = os.path.join(test_img_dir, img_name)

    # Run prediction
    results = custom_model.predict(source=img_path, conf=0.25, verbose=False)

    # Extract the image with boxes drawn on it directly from memory
    res_plotted = results[0].plot()
    res_rgb = cv2.cvtColor(res_plotted, cv2.COLOR_BGR2RGB)

    # Display
    plt.subplot(1, 3, i+1)
    plt.imshow(res_rgb)
    plt.axis('off')
    plt.title(f"Test Image {i+1}")

plt.tight_layout()
plt.show()
```




```
!pip install gymnasium -q
```

```python
import gymnasium as gym
from gymnasium import spaces
import numpy as np
import random

class MockDhakaTrafficEnv(gym.Env):
    def __init__(self):
        super().__init__()

        # ACTIONS: What can the AI do?
        # Action 0: North/South Green (East/West Red)
        # Action 1: East/West Green (North/South Red)
        self.action_space = spaces.Discrete(2)

        # STATE: What does the AI see? (Provided by our YOLO model in
the real world)
        # [North_cars, South_cars, East_cars, West_cars,
Ambulance_Present (0 or 1)]
        low = np.array([0, 0, 0, 0, 0], dtype=np.float32)
        high = np.array([100, 100, 100, 100, 1], dtype=np.float32)
        self.observation_space = spaces.Box(low, high,
dtype=np.float32)

        # Initialize the intersection with a random traffic jam
        self.state = None
        self.reset()

    def reset(self, seed=None, options=None):
        super().reset(seed=seed)
        # Start with a random number of cars in each lane (0 to 40)
        # 10% chance an ambulance is waiting
        ambulance = 1.0 if random.random() < 0.1 else 0.0
        self.state = np.array([
            random.randint(5, 40), # North
            random.randint(5, 40), # South
            random.randint(5, 40), # East
            random.randint(5, 40), # West
            ambulance              # Ambulance
        ], dtype=np.float32)
        return self.state, {}

    def step(self, action):
        # 1. Simulate the cars passing through the Green light
        if action == 0: # North/South Green
            passed_N = min(self.state[0], 10) # Max 10 cars pass per
step
            passed_S = min(self.state[1], 10)
            self.state[0] -= passed_N
            self.state[1] -= passed_S
            cars_passed = passed_N + passed_S
```

```
        else: # East/West Green
            passed_E = min(self.state[2], 10)
            passed_W = min(self.state[3], 10)
            self.state[2] -= passed_E
            self.state[3] -= passed_W
            cars_passed = passed_E + passed_W

        # 2. Add new cars arriving at the red lights (simulating Dhaka
traffic!)
        self.state[0] += random.randint(0, 3)
        self.state[1] += random.randint(0, 3)
        self.state[2] += random.randint(0, 3)
        self.state[3] += random.randint(0, 3)

        # 3. Calculate the Reward (How we train the brain)
        # +1 point for every car that passes
        # -0.5 points for every car still waiting in the total
intersection
        total_waiting = np.sum(self.state[0:4])
        reward = cars_passed - (0.5 * total_waiting)

        # Huge penalty if an ambulance is waiting and the AI ignores
it
        if self.state[4] == 1.0:
            reward -= 50

        # 4. Check if the simulation is "done" (e.g., traffic cleared)
        done = bool(total_waiting == 0)

        # Cap the state values so they don't exceed our 100 max or go
below 0
        self.state = np.clip(self.state, 0, 100)

        return self.state, reward, done, False, {}

print("Mock Dhaka Traffic Environment Successfully Created!")
```

```
Mock Dhaka Traffic Environment Successfully Created!
```

```
# Initialize our mock intersection
env = MockDhakaTrafficEnv()
state, _ = env.reset()

print("--- STARTING RANDOM TRAFFIC LIGHT TEST ---")
print(f"Initial Traffic: North={state[0]}, South={state[1]},
East={state[2]}, West={state[3]}, Ambulance={state[4]}")

total_reward = 0

# Let the dumb agent take 5 random actions (5 time steps)
for step in range(5):
```

```
    # Agent picks a random action (0 or 1)
    action = env.action_space.sample()

    # The environment processes the action
    next_state, reward, done, _, _ = env.step(action)
    total_reward += reward

    action_name = "North/South GREEN" if action == 0 else "East/West
GREEN"
    print(f"\nStep {step+1}: Agent chose {action_name}")
    print(f"Reward Received: {reward}")
    print(f"Current Traffic: North={next_state[0]},
South={next_state[1]}, East={next_state[2]}, West={next_state[3]}")

print(f"\nTotal Score after 5 steps: {total_reward}")
```

```
--- STARTING RANDOM TRAFFIC LIGHT TEST ---
Initial Traffic: North=27.0, South=18.0, East=37.0, West=13.0,
Ambulance=0.0

Step 1: Agent chose North/South GREEN
Reward Received: -20.0
Current Traffic: North=19.0, South=9.0, East=37.0, West=15.0

Step 2: Agent chose North/South GREEN
Reward Received: -13.0
Current Traffic: North=10.0, South=2.0, East=37.0, West=15.0

Step 3: Agent chose North/South GREEN
Reward Received: -17.5
Current Traffic: North=2.0, South=3.0, East=37.0, West=17.0

Step 4: Agent chose North/South GREEN
Reward Received: -26.5
Current Traffic: North=3.0, South=2.0, East=38.0, West=20.0

Step 5: Agent chose East/West GREEN
Reward Received: -4.0
Current Traffic: North=6.0, South=4.0, East=28.0, West=10.0

Total Score after 5 steps: -81.0
```

```
import torch
import torch.nn as nn
import torch.optim as optim
from collections import deque
import random
import numpy as np
from tqdm import tqdm # The progress bar library!
```

```python
# 1. Setup the Device (Use 1 GPU)
device = torch.device("cuda:0" if torch.cuda.is_available() else
"cpu")
print(f"Training on device: {device}")

# 2. Define the Neural Network Architecture
class TrafficBrain(nn.Module):
    def __init__(self, state_size, action_size):
        super(TrafficBrain, self).__init__()
        self.fc1 = nn.Linear(state_size, 64)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(64, 64)
        self.fc3 = nn.Linear(64, action_size)

    def forward(self, x):
        out = self.fc1(x)
        out = self.relu(out)
        out = self.fc2(out)
        out = self.relu(out)
        return self.fc3(out)

# 3. Setup the Agent and Hyperparameters
state_size = 5
action_size = 2

learning_rate = 0.001
gamma = 0.95
epsilon = 1.0
epsilon_min = 0.05
epsilon_decay = 0.995

# Move model to the GPU
model = TrafficBrain(state_size, action_size).to(device)
optimizer = optim.Adam(model.parameters(), lr=learning_rate)
criterion = nn.MSELoss()
memory = deque(maxlen=2000)

# 4. The Training Loop with tqdm
episodes = 500
batch_size = 32
scores = []

env = MockDhakaTrafficEnv()

# The tqdm progress bar wraps our episodes range!
for e in tqdm(range(episodes), desc="Training RL Agent"):
    state, _ = env.reset()
    total_reward = 0

    for time_step in range(20):
```

```
        # Move state to GPU for prediction
        state_tensor =
torch.FloatTensor(state).unsqueeze(0).to(device)

        if random.random() <= epsilon:
            action = env.action_space.sample()
        else:
            with torch.no_grad():
                q_values = model(state_tensor)
                action = torch.argmax(q_values).item()

        next_state, reward, done, _, _ = env.step(action)
        total_reward += reward

        # Save to memory (keep as numpy arrays to save GPU memory)
        memory.append((state, action, reward, next_state, done))
        state = next_state

        # D. Learn from past memory (Vectorized for GPU acceleration!)
        if len(memory) > batch_size:
            batch = random.sample(memory, batch_size)

            # Convert batch to PyTorch tensors and move to GPU
            b_states = torch.FloatTensor(np.array([b[0] for b in
batch])).to(device)
            b_actions = torch.LongTensor(np.array([b[1] for b in
batch])).unsqueeze(1).to(device)
            b_rewards = torch.FloatTensor(np.array([b[2] for b in
batch])).unsqueeze(1).to(device)
            b_next_states = torch.FloatTensor(np.array([b[3] for b in
batch])).to(device)
            b_dones = torch.FloatTensor(np.array([b[4] for b in
batch])).unsqueeze(1).to(device)

            # Current Q Values
            curr_q = model(b_states).gather(1, b_actions)

            # Max Next Q Values (Bellman Equation)
            with torch.no_grad():
                max_next_q = model(b_next_states).max(1)
[0].unsqueeze(1)
                expected_q = b_rewards + gamma * max_next_q * (1 -
b_dones)

            # Backpropagation
            loss = criterion(curr_q, expected_q)
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()
```

```
        if done:
            break

    if epsilon > epsilon_min:
        epsilon *= epsilon_decay

    scores.append(total_reward)

print("\nTraining Complete!")
print(f"Final Average Score (last 50 episodes): {np.mean(scores[-
50:]):.1f}")
```

```
Training on device: cuda:0

Training RL Agent: 100%|██████████| 500/500 [00:38<00:00, 13.02it/s]


Training Complete!
Final Average Score (last 50 episodes): -177.1
```

```
print("--- TESTING THE TRAINED AI BRAIN ---")
state, _ = env.reset()
total_reward = 0

# We will let the smart agent control the lights for 5 steps
for step in range(5):
    # 1. Print what the AI sees
    north, south, east, west, amb = state
    amb_status = "YES (EMERGENCY!)" if amb == 1.0 else "NO"
    print(f"\n[Step {step+1}] Traffic Queue: North={int(north)} |
South={int(south)} | East={int(east)} | West={int(west)} |
Ambulance={amb_status}")

    # 2. Ask the trained Neural Network what to do
    # Notice there is no 'epsilon' here. We are 100% EXPLOITING the
trained brain.
    state_tensor = torch.FloatTensor(state).unsqueeze(0).to(device)
    with torch.no_grad():
        q_values = model(state_tensor)
        action = torch.argmax(q_values).item()

    action_name = "North/South GREEN" if action == 0 else "East/West
GREEN"
    print(f"-> AI Decision: Turn {action_name}!")
    print(f"   (Q-Values calculated by Brain: N/S={q_values[0]
[0]:.2f}, E/W={q_values[0][1]:.2f})")

    # 3. Step the environment forward
```

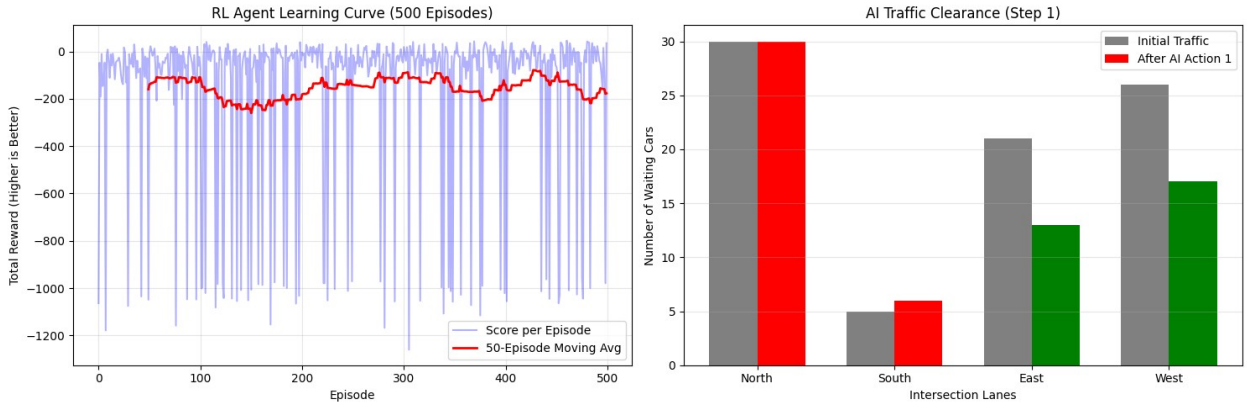
```
    state, reward, done, _, _ = env.step(action)
    total_reward += reward

print(f"\nFinal Score after 5 steps: {total_reward:.1f}")
```

```
--- TESTING THE TRAINED AI BRAIN ---

[Step 1] Traffic Queue: North=32 | South=7 | East=24 | West=38 |
Ambulance=NO
-> AI Decision: Turn East/West GREEN!
   (Q-Values calculated by Brain: N/S=4.64, E/W=19.70)

[Step 2] Traffic Queue: North=33 | South=8 | East=16 | West=28 |
Ambulance=NO
-> AI Decision: Turn East/West GREEN!
   (Q-Values calculated by Brain: N/S=24.72, E/W=33.45)

[Step 3] Traffic Queue: North=36 | South=9 | East=7 | West=18 |
Ambulance=NO
-> AI Decision: Turn East/West GREEN!
   (Q-Values calculated by Brain: N/S=39.95, E/W=43.50)

[Step 4] Traffic Queue: North=38 | South=9 | East=2 | West=9 |
Ambulance=NO
-> AI Decision: Turn East/West GREEN!
   (Q-Values calculated by Brain: N/S=56.14, E/W=56.35)

[Step 5] Traffic Queue: North=41 | South=12 | East=3 | West=0 |
Ambulance=NO
-> AI Decision: Turn North/South GREEN!
   (Q-Values calculated by Brain: N/S=48.98, E/W=46.48)

Final Score after 5 steps: -68.0
```

```
import matplotlib.pyplot as plt
import numpy as np
import torch

# ==========================================
# VISUALIZATION 1: THE LEARNING CURVE
# ==========================================
plt.figure(figsize=(15, 5))

# Plot 1: AI Training Progress
plt.subplot(1, 2, 1)
# Calculate a moving average to smooth out the noisy RL chart
moving_avg = np.convolve(scores, np.ones(50)/50, mode='valid')

plt.plot(scores, label='Score per Episode', alpha=0.3, color='blue')
plt.plot(range(49, len(scores)), moving_avg, label='50-Episode Moving
Avg', color='red', linewidth=2)
```

```
plt.title('RL Agent Learning Curve (500 Episodes)')
plt.xlabel('Episode')
plt.ylabel('Total Reward (Higher is Better)')
plt.legend()
plt.grid(True, alpha=0.3)

# ==========================================
# VISUALIZATION 2: TRAFFIC CLEARANCE (INFERENCE)
# ==========================================
plt.subplot(1, 2, 2)
state, _ = env.reset()
lanes = ['North', 'South', 'East', 'West']

# Let the AI take 4 steps and record the history
history_states = [state[:4]] # Only taking the car counts, ignoring
ambulance for the plot
colors = []

for step in range(4):
    state_tensor = torch.FloatTensor(state).unsqueeze(0).to(device)
    with torch.no_grad():
        action = torch.argmax(model(state_tensor)).item()

    # Record which lights were green (0 = N/S Green, 1 = E/W Green)
    if action == 0:
        colors.append(['green', 'green', 'red', 'red'])
    else:
        colors.append(['red', 'red', 'green', 'green'])

    state, _, _, _, _ = env.step(action)
    history_states.append(state[:4])

# Plotting the before and after of the first 2 steps as a
stacked/grouped bar chart
x = np.arange(len(lanes))
width = 0.35

# Step 0 (Initial Traffic) vs Step 1 (After 1st AI action)
plt.bar(x - width/2, history_states[0], width, label='Initial
Traffic', color='gray')
plt.bar(x + width/2, history_states[1], width, label='After AI Action
1', color=colors[0])

plt.title('AI Traffic Clearance (Step 1)')
plt.xlabel('Intersection Lanes')
plt.ylabel('Number of Waiting Cars')
plt.xticks(x, lanes)
plt.legend()
plt.grid(axis='y', alpha=0.3)
```

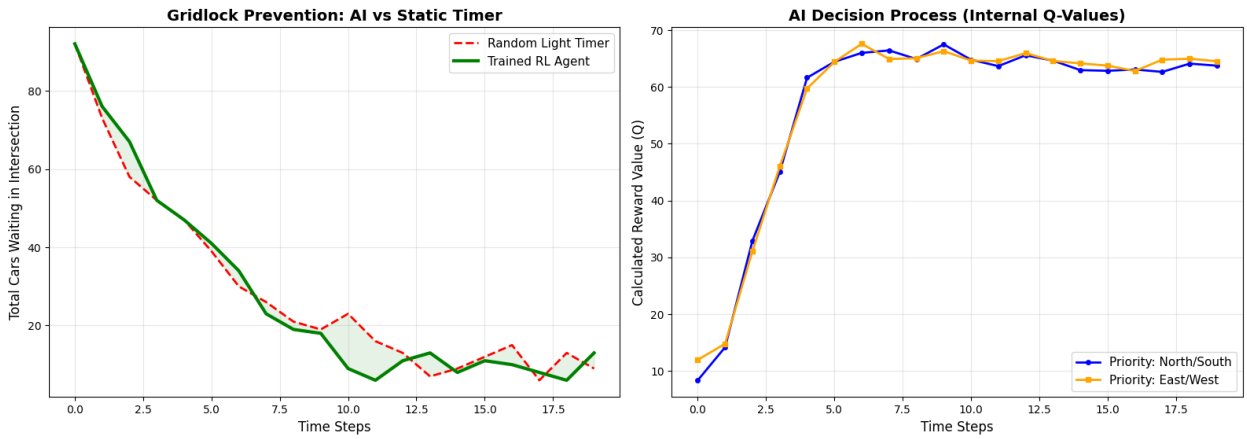
```
plt.tight_layout()
plt.show()
```




```
import matplotlib.pyplot as plt
import numpy as np
import torch
import random

# ==========================================
# SIMULATION SETUP (FAIR COMPARISON)
# ==========================================
# We use a fixed seed so BOTH agents experience the exact same
incoming traffic!
fixed_seed = 42

# --- Run 1: The Dumb/Random Agent ---
random.seed(fixed_seed)
env_random = MockDhakaTrafficEnv()
state, _ = env_random.reset()
initial_state = state.copy() # Save this exact traffic jam for the
Smart AI

random_waiting_history = []
for step in range(20):
    total_waiting = np.sum(state[:4])
    random_waiting_history.append(total_waiting)

    action = env_random.action_space.sample() # Random light changes
    state, _, _, _, _ = env_random.step(action)

# --- Run 2: The Smart Trained AI ---
random.seed(fixed_seed)
env_smart = MockDhakaTrafficEnv()
env_smart.state = initial_state.copy() # Force the exact same starting
traffic
state = env_smart.state
```

```
smart_waiting_history = []
q_history_ns = []
q_history_ew = []

for step in range(20):
    total_waiting = np.sum(state[:4])
    smart_waiting_history.append(total_waiting)

    state_tensor = torch.FloatTensor(state).unsqueeze(0).to(device)
    with torch.no_grad():
        q_values = model(state_tensor)
        action = torch.argmax(q_values).item()

        # Record the brain's internal Q-values
        q_history_ns.append(q_values[0][0].item())
        q_history_ew.append(q_values[0][1].item())

    state, _, _, _, _ = env_smart.step(action)


# ==========================================
# PLOTTING THE RESULTS
# ==========================================
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))

# Plot 1: The Proof (Smart vs Random)
ax1.plot(random_waiting_history, label='Random Light Timer',
color='red', linestyle='--', linewidth=2)
ax1.plot(smart_waiting_history, label='Trained RL Agent',
color='green', linewidth=3)
ax1.fill_between(range(20), random_waiting_history,
smart_waiting_history, color='green', alpha=0.1)

ax1.set_title('Gridlock Prevention: AI vs Static Timer', fontsize=14,
fontweight='bold')
ax1.set_xlabel('Time Steps', fontsize=12)
ax1.set_ylabel('Total Cars Waiting in Intersection', fontsize=12)
ax1.legend(loc='upper right', fontsize=11)
ax1.grid(True, alpha=0.3)
ax1.text(0.5, -0.15, "The green shaded area represents the traffic
jams prevented by the AI.",
         ha='center', va='center', transform=ax1.transAxes,
fontsize=10, fontstyle='italic')

# Plot 2: The Brain Waves (Q-Values)
ax2.plot(q_history_ns, label='Priority: North/South', color='blue',
linewidth=2, marker='o', markersize=4)
ax2.plot(q_history_ew, label='Priority: East/West', color='orange',
linewidth=2, marker='s', markersize=4)
```

```
ax2.set_title('AI Decision Process (Internal Q-Values)', fontsize=14,
fontweight='bold')
ax2.set_xlabel('Time Steps', fontsize=12)
ax2.set_ylabel('Calculated Reward Value (Q)', fontsize=12)
ax2.legend(loc='lower right', fontsize=11)
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



The green shaded area represents the traffic jams prevented by the AI.